

UNIVERSITY FOR THE CREATIVE ARTS
Berlin School of Business & Innovation (BSBI)

MSc in Artificial Intelligence

Module: Introduction to Artificial Intelligence

Predictive Modelling Using Machine Learning

Predicting Credit Card Payment Default with Interpretable Machine Learning

Practical Skills Assessment

Student: Osama Al-Momani

Issued by: Dr. Lawrence Ibeh

Submission Date: 16/09/2026

Word count (body text, excl. references): ~1,500

1. Introduction

Credit card default prediction is a well-established application of machine learning in financial risk management: identifying, ahead of time, which clients are likely to miss a payment allows a lender to intervene, adjust credit limits, or price risk more accurately (Yeh and Lien, 2009). This report documents an end-to-end predictive modelling pipeline built on the Credit_Card.csv dataset (34,788 clients, 30 features, April–September 2005), covering exploratory data analysis, data cleaning, model training, evaluation and interpretation, and a critical discussion of the pipeline's limitations.

2. Literature Review

Credit scoring is a heavily benchmarked classification task. Lessmann et al. (2015) compared over 40 classifiers across multiple credit-scoring datasets and found ensembles such as random forests consistently outperformed single classifiers and logistic regression on AUC, while remaining more robust to noisy features than distance-based methods like KNN. Breiman (2001) proposed random forests as a variance-reduction technique over single decision trees, motivating their inclusion here. XGBoost (Chen and Guestrin, 2016) is a de-facto standard for tabular tasks, offering strong performance with efficient training. Because tree ensembles are not directly interpretable, this report follows Lundberg and Lee (2017), whose SHAP method attributes individual predictions to input features and is used here to interpret the tuned best model.

3. Methodology

The pipeline was implemented in Python (pandas, scikit-learn, XGBoost, SHAP) and structured into five stages mirroring the assignment tasks: (1) EDA, (2) data preparation, (3) model training, (4) evaluation and tuning, and (5) critical reflection. All code is included in the accompanying repository and is fully commented and reproducible (see README.md).

4. Results and Discussion

4.1 Task 1 — Exploratory Data Analysis

Class imbalance. The target variable is imbalanced: 80.9% of clients did not default versus 19.1% who did (28,132 vs 6,656 clients). This 4.2:1 ratio is moderate rather than severe, but it is large enough that accuracy alone is a poor performance criterion, and it motivated the use of stratified sampling and AUC-ROC as the primary ranking metric in Task 4.

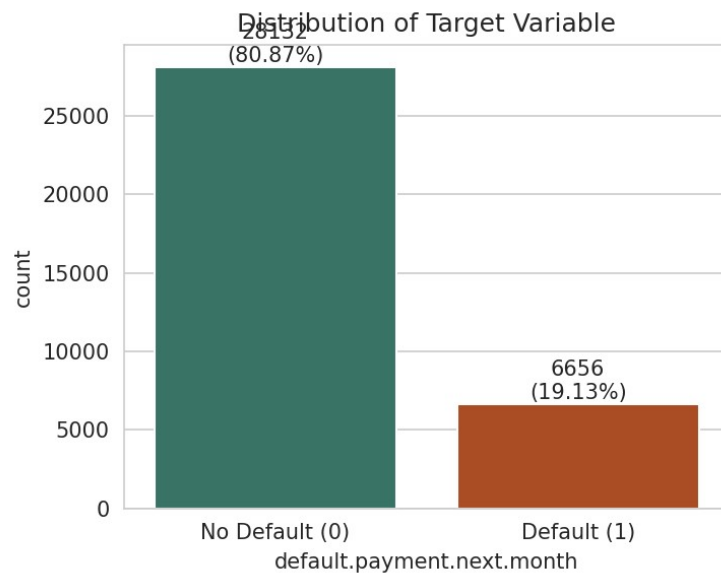


Figure 1. Class distribution of default.payment.next.month.

Descriptive statistics for the key numerical features are summarised in Table 1. Credit limits (LIMIT_BAL) are strongly right-skewed (mean NT\$182,475 vs median NT\$140,000), which is why the log-transformed LIMIT_BAL_LOG feature is useful for linear models. Cumulative billing (BILL_AMT_SUM) is similarly skewed with a small number of very large or negative (over-paid) balances.

Feature	Mean	Median	Std. Dev.	Min	Max
LIMIT_BAL	182474.58	140000.0	230266.05	10000.0	5600000.0
AGE	35.44	34.0	9.01	21.0	79.0
BILL_AMT_SUM	270324.52	127192.5	379137.33	-336259.0	5263883.0
LIMIT_BAL_LOG	11.71	11.85	0.94	9.21	15.54
risk_leak	0.19	0.03	0.41	-0.39	1.4

Table 1. Descriptive statistics for key numeric features.

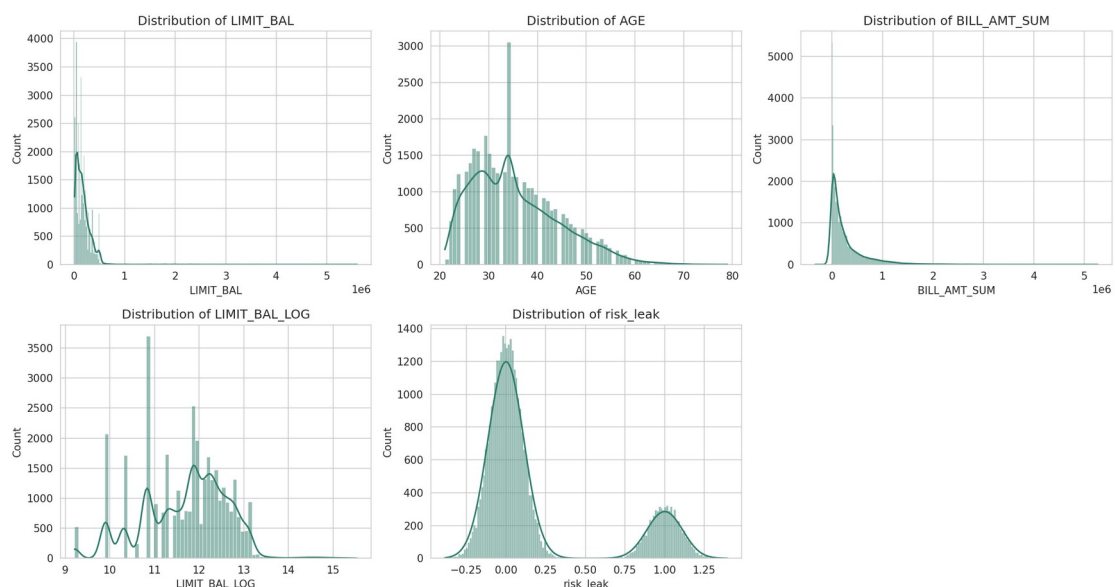


Figure 2. Distributions of key numeric features.

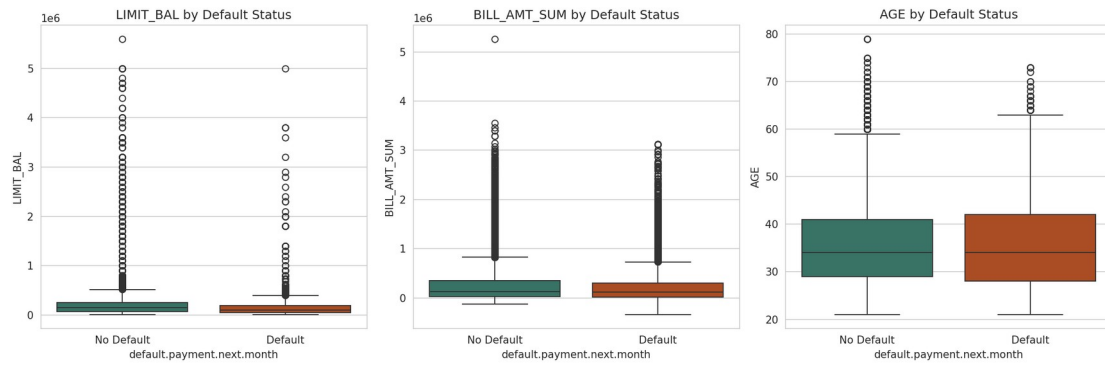


Figure 3. LIMIT_BAL, BILL_AMT_SUM and AGE by default status, showing outliers and the (weak) separation by AGE and LIMIT_BAL.

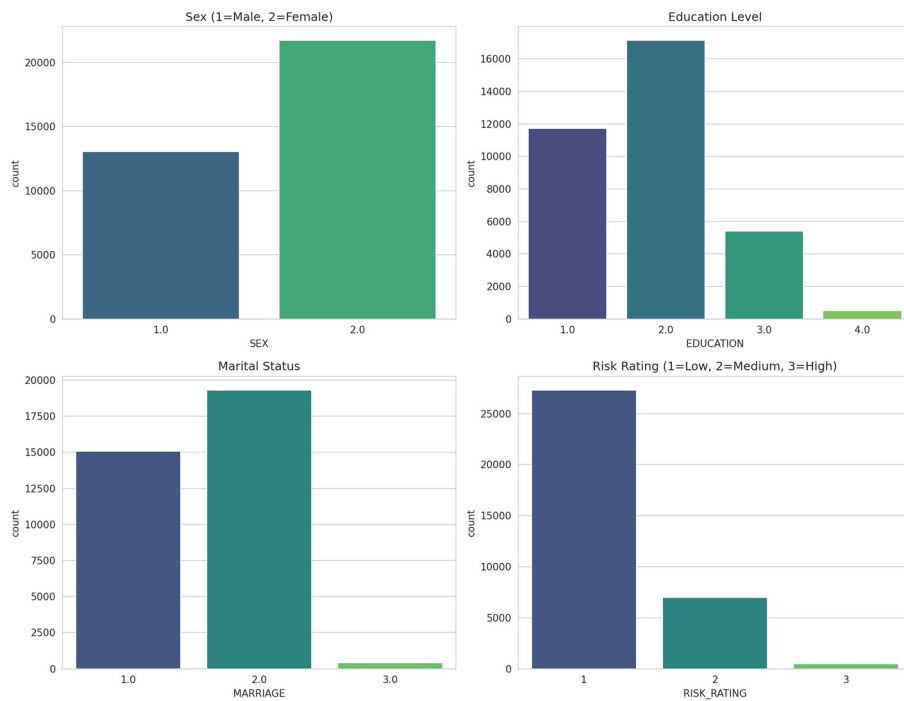


Figure 4. Distribution of SEX, EDUCATION, MARRIAGE and RISK_RATING.

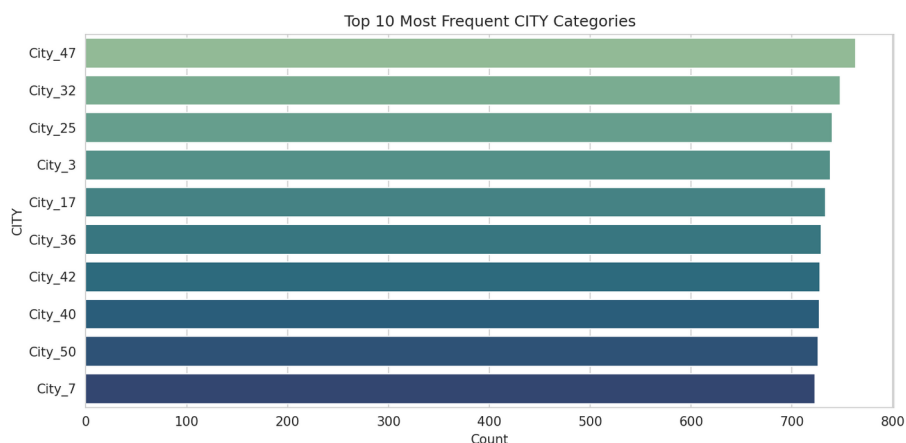


Figure 5. The 50 CITY categories are near-uniformly distributed (top 10 shown); city alone carries little signal.

Correlation with the target and a critical finding. A correlation analysis (Figure 6, Figure 7) revealed that most engineered/behavioural variables correlate weakly to moderately with default: RISK_RATING ($r=0.36$) and the repayment-status columns PAY_0 through PAY_6 ($r=0.18$ – 0.31 , decaying with recency) are the strongest legitimate predictors, consistent with the intuition that recent delinquency is the best predictor of future default. However, risk_leak correlates at $r=0.97$ with the target — far higher than any behavioural or

demographic feature, and implausible for an independently observed "risk score". Combined with its name, this is a clear symptom of target leakage. A short demonstration (see `task3b_leakage_demo.py`) confirms this: a plain Logistic Regression trained with `risk_leak` included achieves 100% accuracy, precision, recall and AUC-ROC on the held-out test set — a result with no practical value, since it simply re-derives the label rather than predicting it. `risk_leak` was therefore excluded from the modelling pipeline in Tasks 3–4; this decision, and its implications, are revisited in Section 5.

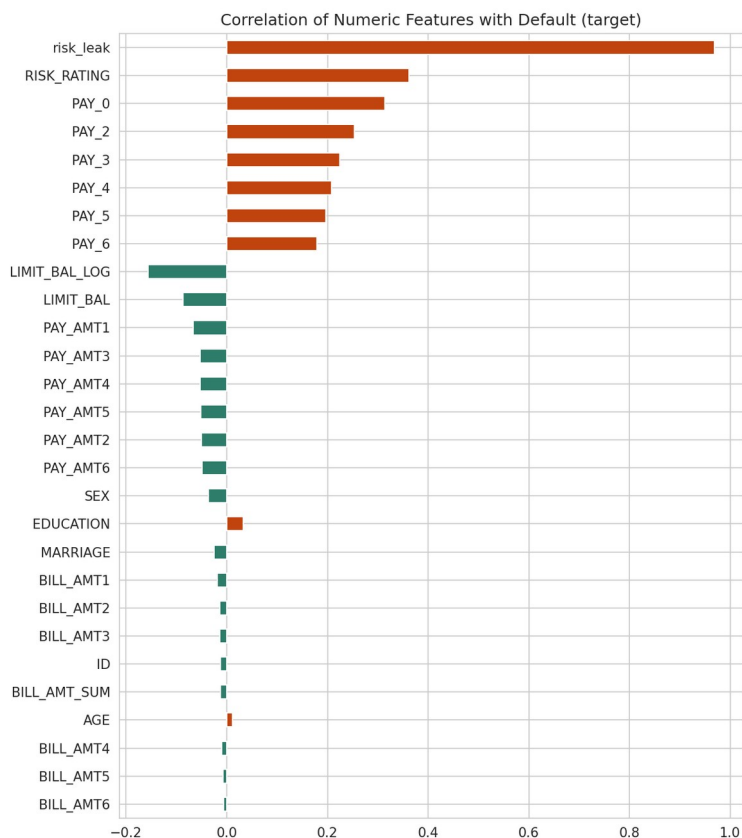


Figure 6. Correlation of numeric features with the target (`risk_leak` omitted from later modelling).

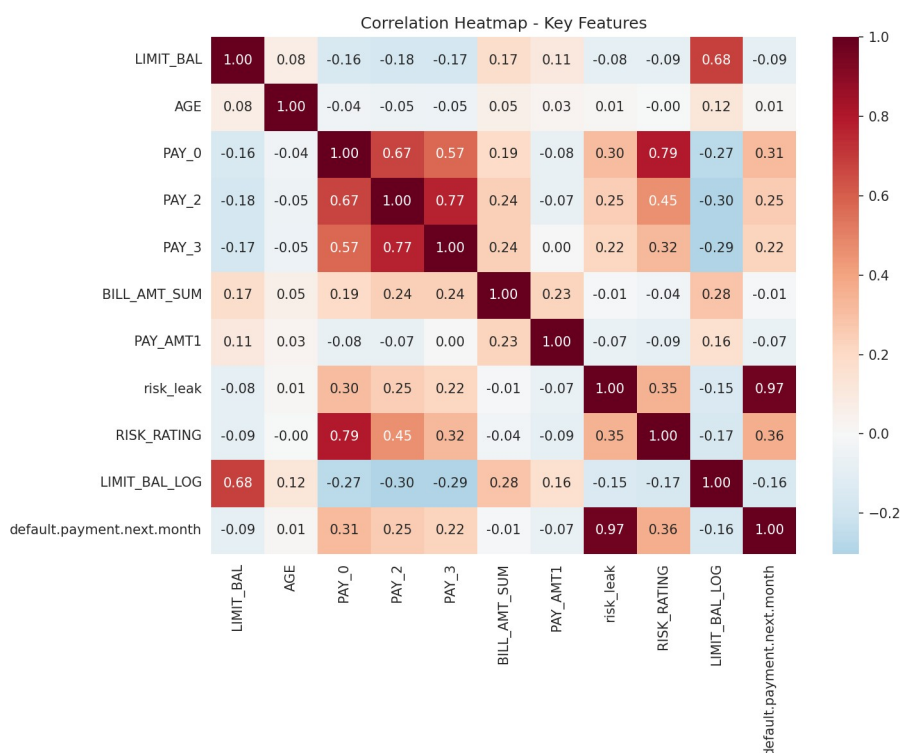


Figure 7. Correlation heatmap for key features, showing `risk_leak`'s anomalous 0.97 correlation with the target.

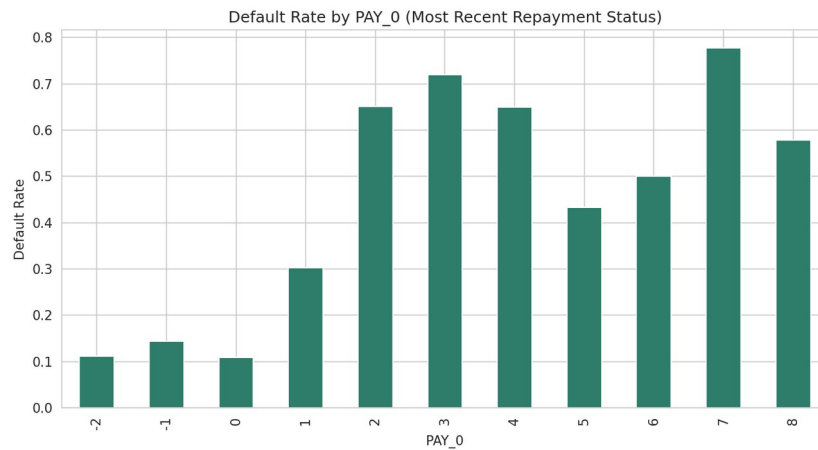


Figure 8. Default rate rises sharply with worsening PAY_0 (repayment delay), confirming it as a genuine predictor.



Figure 9. Default rate increases monotonically with RISK_RATING.

4.2 Task 2 — Data Preparation

The raw CSV contained two distinct data-quality problems beyond ordinary missing values. First, LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE, PAY_AMT1 and PAY_AMT2 each had 1,700–1,800 genuine missing values (~5% of rows); these were imputed using the median (continuous features) or mode (categorical-coded features), which is robust to the skew observed in Task 1. Second, risk_leak and LIMIT_BAL_LOG were corrupted by a digit-grouping fault: the true decimal point had been replaced with periods re-inserted every three characters (e.g. the true value 1.075675276433290 was stored as the string "1.075.675.276.433.290"), affecting 3,387 risk_leak values and effectively all LIMIT_BAL_LOG values. Because LIMIT_BAL_LOG and BILL_AMT_SUM are deterministic transforms of other columns ($\ln(\text{LIMIT_BAL})$ and the sum of BILL_AMT1–6 respectively), both were recomputed directly rather than reverse-engineering the corrupted strings, which is a safer and fully auditable fix; 333 additional silent mismatches in BILL_AMT_SUM were corrected the same way. The 31 risk_leak values that used an unrelated scientific-notation-with-comma corruption (e.g. "-2,00E+11", far outside the documented [-0.39, 1.40] range) were treated as unrecoverable and set to missing, then median-imputed.

CITY (50 near-uniform categories, Figure 5) was frequency-encoded rather than one-hot encoded, avoiding 49 additional sparse columns while preserving any city-level signal. SEX, EDUCATION, MARRIAGE and RISK_RATING were retained as their original integer codes; undocumented EDUCATION codes (0, 5, 6) and MARRIAGE code 0 were collapsed into an explicit "Other" category. All numeric features were standardised with scikit-learn's StandardScaler (zero mean, unit variance), which is required by distance- and gradient-based models (KNN, SVM, Logistic Regression) and does not harm tree-based models. Table 2 shows a before/after example for four representative features.

Row	LIMIT_BAL	AGE	BILL_AMT_SUM
Before (1)	20,000	24	7,704
After (1)	-0.706	-1.270	-0.693
Before (2)	120,000	26	17,077
After (2)	-0.271	-1.048	-0.668

Table 2. Before/after example of scaling (StandardScaler) on two sample rows.

After cleaning, 27 features were retained for modelling (risk_leak excluded, see Section 4.1) across 34,788 rows, with zero remaining missing values.

4.3 Task 3 — Model Training

The data was split 80/20 into training (27,830 rows) and test (6,958 rows) sets using stratified sampling on the target, preserving the 80.9%/19.1% class ratio in both splits. An 80/20 ratio was chosen to keep a large enough test set (~7,000 rows) for a statistically stable estimate of AUC-ROC and F1-score, while retaining ample training data for the more data-hungry models (Random Forest, XGBoost).

Six classifiers were trained with default hyperparameters: Logistic Regression (linear baseline), SVM with an RBF kernel (non-linear boundaries), Decision Tree (interpretable baseline for the ensembles), Random Forest and XGBoost (ensembles expected to handle mixed numeric/ordinal features and class imbalance well, per Lessmann et al., 2015), and KNN (non-parametric baseline). All initial hyperparameters were recorded before tuning (full listing in task3_initial_hyperparameters.json, e.g. Random Forest: n_estimators=100, max_depth=None, criterion=gini). Because SVM training scales poorly (roughly $O(n^2)$ – $O(n^3)$) with training-set size, it was trained on a stratified 6,000-row subsample rather than the full 27,830 rows — a documented practical compromise discussed further in Section 5.

4.4 Task 4 — Model Evaluation and Visualisation

Table 3 compares all six models on the held-out test set. Because of the class imbalance identified in Task 1, models are ranked by AUC-ROC rather than F1-score or accuracy, since AUC-ROC is insensitive to the default 0.5 decision threshold and to the class-balance-driven accuracy inflation visible here (every model exceeds 82% accuracy simply by leaning on the majority class).

Model	Accuracy	Precision	Recall	F1-score	AUC-ROC
Random Forest	0.8583	0.8165	0.3343	0.4744	0.8703
XGBoost	0.8528	0.7563	0.3403	0.4694	0.8005
Logistic Regression	0.8212	0.5848	0.2254	0.3254	0.7509
K-Nearest Neighbors	0.8306	0.605	0.3291	0.4263	0.7299
Support Vector Machine	0.8389	0.6756	0.3035	0.4189	0.7167
Decision Tree	0.8372	0.6199	0.3847	0.4747	0.6644

Table 3. Test-set performance of all six models (default hyperparameters), ranked by AUC-ROC.

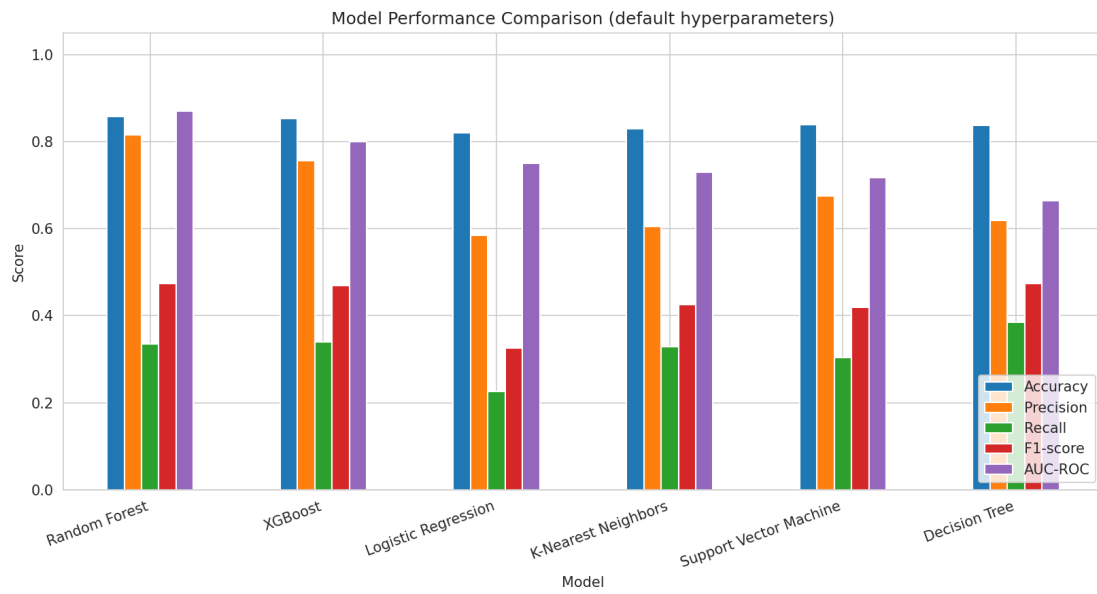


Figure 10. Side-by-side comparison of all five metrics across models.

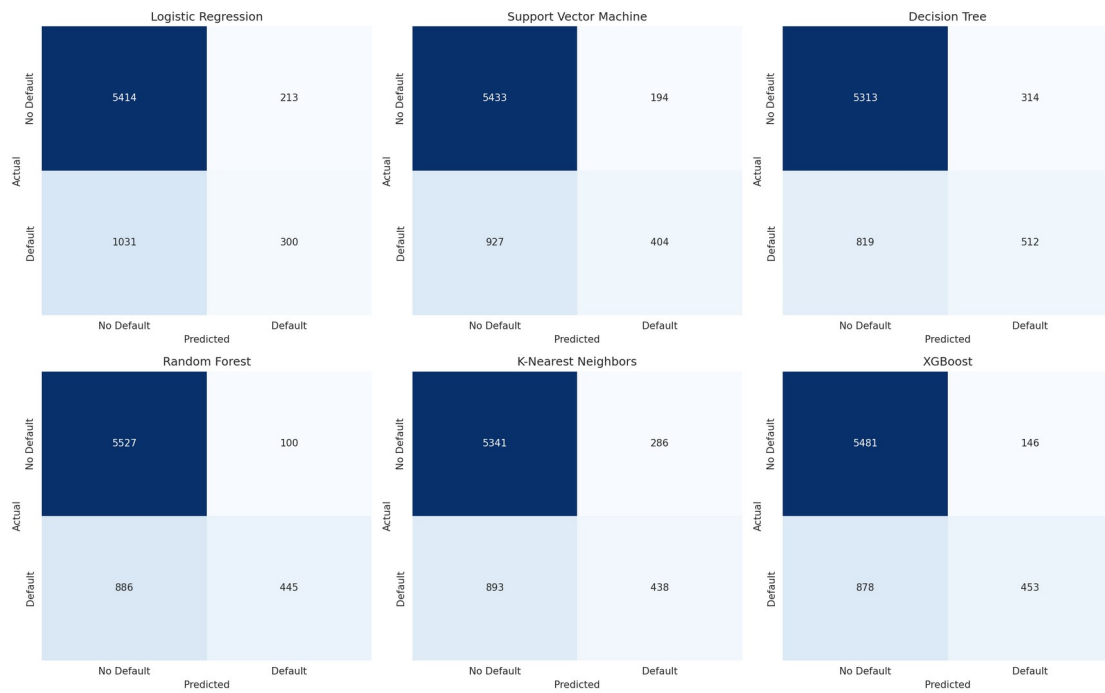


Figure 11. Confusion matrices for all six models.

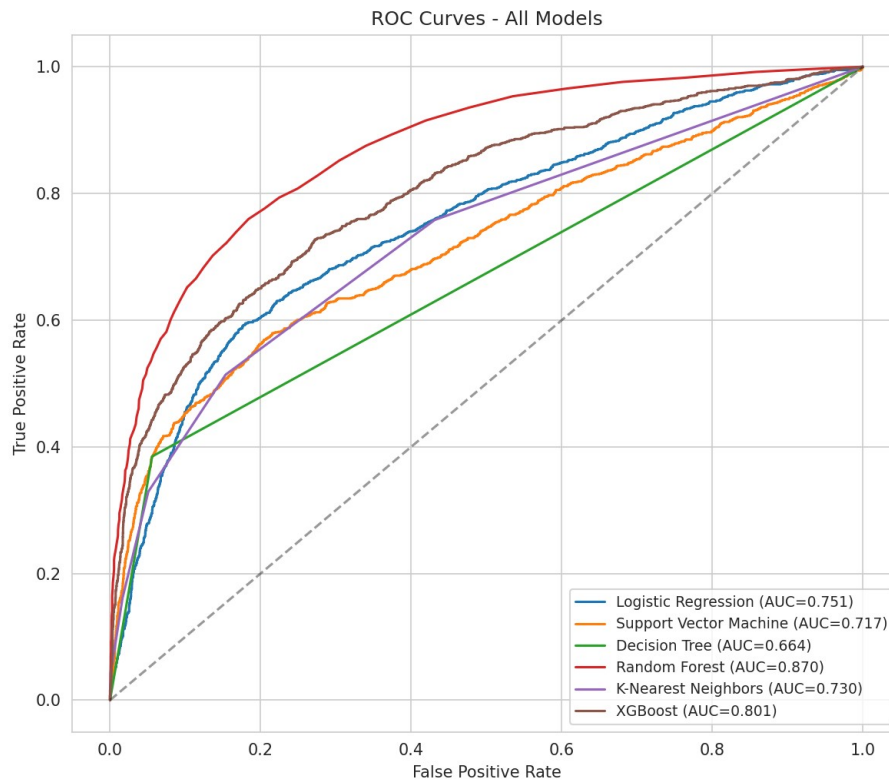


Figure 12. ROC curves for all six models; Random Forest dominates across all thresholds.

Random Forest was the clear best performer (AUC-ROC 0.870, Precision 0.817), ahead of XGBoost (0.801) and every other model; it was selected for tuning via GridSearchCV (3-fold CV, optimising F1-score) over `n_estimators`, `max_depth` and `min_samples_split`. Table 4 compares performance before and after tuning.

Metric	Before Tuning	After Tuning
Accuracy	0.8583	0.8599
Precision	0.8165	0.8333
Recall	0.3343	0.3343
F1-score	0.4744	0.4772
AUC-ROC	0.8703	0.8552

Table 4. Random Forest before vs after tuning. Best parameters found: `max_depth=None`, `min_samples_split=5`, `n_estimators=100`.

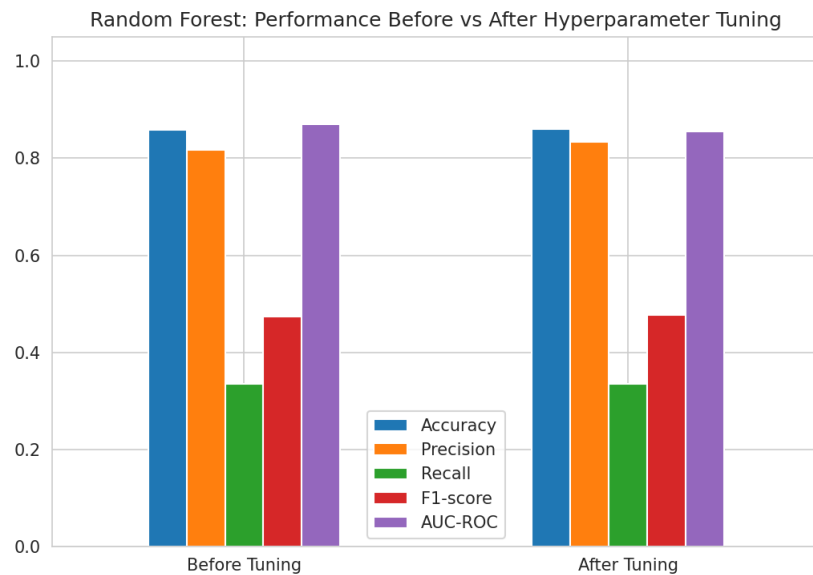


Figure 13. Random Forest performance before vs after GridSearchCV tuning.

Tuning improved precision (0.817→0.833) and F1-score (0.474→0.477) marginally, but AUC-ROC fell slightly (0.870→0.855) — an honest result: GridSearchCV optimised for F1-score at the default 0.5 threshold, a different objective from the threshold-independent AUC-ROC, so a small trade-off is expected rather than a sign of a faulty pipeline. The untuned model suits ranking clients by risk; the tuned model suits 0.5-threshold classification.

SHAP (TreeExplainer) was used to interpret the tuned Random Forest on a 200-row sample of the test set (Figure 14, Table 5). PAY_0 (most recent repayment status) is overwhelmingly the most influential feature, followed by RISK_RATING and PAY_2, confirming the correlation analysis from Task 1: recent repayment behaviour and the bank's own risk classification are the dominant legitimate drivers of predicted default, while individual monthly bill and payment amounts contribute comparatively little on their own.

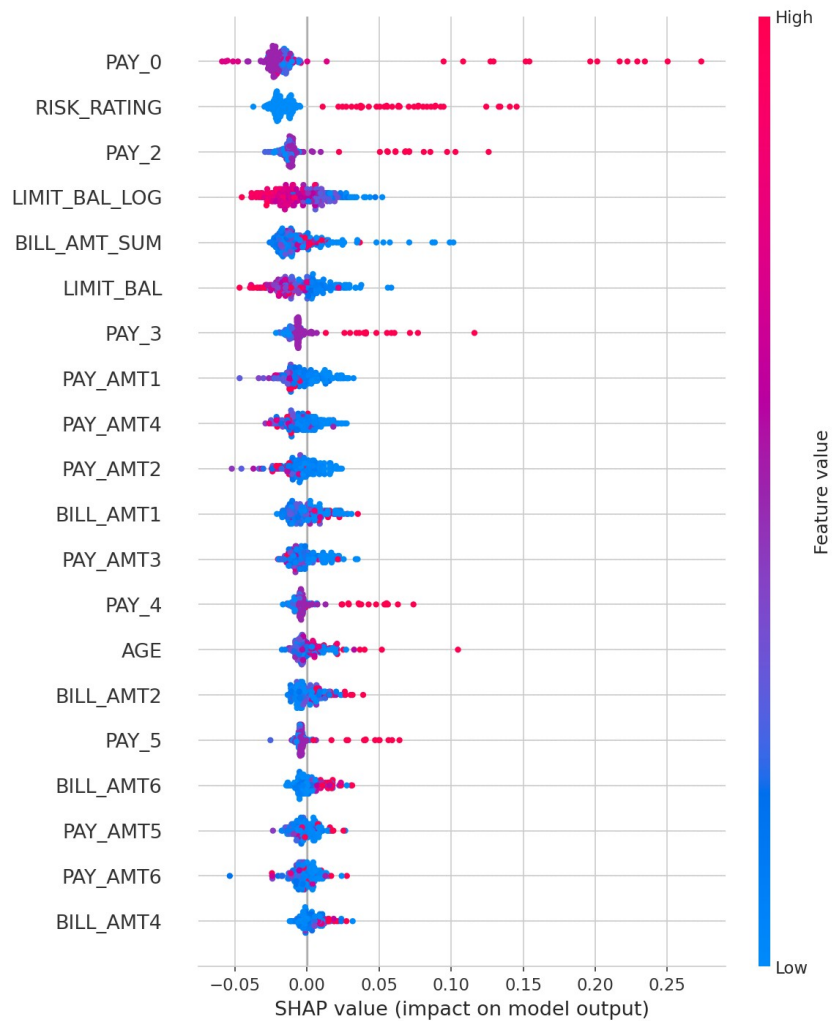


Figure 14. SHAP summary plot for the tuned Random Forest (200-row test sample). Red = high feature value; points to the right increase predicted default risk.

Feature	Mean SHAP value
PAY_0	0.0317
RISK_RATING	0.0266
PAY_2	0.0172
LIMIT_BAL_LOG	0.0154
BILL_AMT_SUM	0.0146
LIMIT_BAL	0.0143
PAY_3	0.0118
PAY_AMT1	0.0111

Table 5. Top 8 features by mean absolute SHAP value.

5. Conclusion and Future Work

5.1 Summary of Findings

Across six classifiers on a realistic (leakage-free) feature set, Random Forest was best-performing (AUC-ROC 0.870 untuned, 0.855 tuned), ahead of XGBoost, SVM, KNN, Logistic Regression and a single Decision Tree — consistent with the credit-scoring literature's finding that ensembles outperform single classifiers and linear baselines on tabular financial data (Lessmann et al., 2015). SHAP showed recent repayment status (PAY_0,

PAY_2) and RISK_RATING are the dominant legitimate predictors, while raw bill/payment amounts and demographics contribute much less — a sensible, explainable result for a credit-risk use case.

5.2 Limitations

- Data leakage (risk_leak): as shown in Section 4.1, this feature almost perfectly encodes the label and was excluded from modelling; its presence in the raw dataset is a reminder that correlation analysis and domain scrutiny must precede modelling, not follow it.
- Missing values: ~5% missingness in seven columns was imputed with median/mode rather than a model-based method (e.g. MICE); this is simple and auditable but slightly understates the true uncertainty in those rows and could bias estimates if missingness is not random.
- Demographic bias: SEX, EDUCATION and MARRIAGE were retained as predictors. Although their individual correlation with default is low ($|r| \leq 0.04$), a deployed model should be audited for disparate impact across these groups before use in real lending decisions, since even indirect correlation via other features could reproduce historical bias.
- Computational constraints: SVM was trained on a 6,000-row subsample rather than the full training set, and GridSearchCV was run on a reduced grid, both due to the single-core evaluation environment used for this assessment; a full grid on complete data may shift the tuned results slightly.

5.3 Future Work

- Investigate and, if appropriate, remove RISK_RATING alongside risk_leak: RISK_RATING's comparatively high correlation with default ($r=0.36$) is defensible if it reflects a legitimate pre-existing bank risk score, but this should be confirmed with the data owner rather than assumed, since an undocumented derived variable was already found to leak the label once.
- Apply class-imbalance-aware training (e.g. class_weight='balanced', SMOTE, or focal loss) to address the observed low recall (0.33–0.38 across models): the current models miss roughly two-thirds of true defaulters, which is the costlier error type for a lender and was not directly optimised for here.
- Run a full, unrestricted GridSearchCV/RandomizedSearchCV across all six models on complete (non-subsampled) data using a distributed or GPU-accelerated environment, to verify that the Random Forest vs XGBoost ranking is stable under exhaustive tuning rather than an artefact of the reduced grid used here.

References

Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32.

Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.

Lessmann, S., Baesens, B., Seow, H.V. and Thomas, L.C. (2015) 'Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research', European Journal of Operational Research, 247(1), pp. 124–136.

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, 30.

Yeh, I.C. and Lien, C.H. (2009) 'The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients', Expert Systems with Applications, 36(2), pp. 2473–2480.

Appendix A: Code and Replication

Full, commented Python source code (data_prep.py, task1_eda.py, task2_prepare.py, task3_train.py, task3b_leakage_demo.py, task4_evaluate.py), a requirements.txt, and this report are provided together in the submitted zip archive. Replication instructions are in the accompanying README.md.